DATA ITEM DESCRIPTION
Title: NUCLEAR SAFETY SOFTWARE VERIFICATION REPORT

Number: DI-NUOR-82430A
AMSC Number: F10534
DTIC Applicable: No
Preparing Activity: 37 (AFSEC/SEWN)
Applicable Forms: N/A

Approval Date: 20250219
Limitation: N/A
GIDEP Applicable: No
Project Number: NUOR-2025-001

Use/Relationship: Following the purpose, methods, and procedures the Nuclear Safety Software Verification Plan (NSSVP) defines, the Nuclear Safety Software Verification Report(s) (NSSVR) provides evidence and documentation of the verification effort as part of Nuclear Safety Design Certification (NSDC). Air Force Safety Center Nuclear Weapons Safety Branch (AFSEC/SEWN) provides NSDC to system program offices for specific nuclear weapon systems and nuclear weapon system modifications.

Chapter 4 of Department of the Air Force Manual (DAFMAN) 91- 119, Safety Design and Evaluation Criteria for Nuclear Weapon Systems Software describes the NSSVR. DAFMAN 91-119 implements Department of the Air Force Instruction (DAFI) 91-101, Air Force Nuclear Weapons Surety Program. DAFI 91-101 explains the process for NSDC. Both DAFMAN 91-119 and DAFI 91-101 are available without restriction on the Air Force E-Publishing website, https://www.e-publishing.af.mil/.

a. This Data Item Description (DID) applies to the acquisition of software for nuclear weapon systems that must be nuclear safety design certified per DAFI 91-101. Per Chapter 2 of DAFMAN 91-119, software is "…application software that executes sequentially and reconfigurable logic implementations that use a hardware description language for development."

b. DAFMAN 91-119 and DAFI 91-101 defines the information required to implement this DID. This information includes the following:

- Procedures and plans that assure independence between the Development Organization (DO) and the Independent Verification Organization (IVO);
- Compensatory measures the IVO took to address any concerns of independence between the DO and IVO;
- Verification tools, methodologies, configuration, and procedures the IVO used in performing the verification effort;
- Traceability between Nuclear Safety Objectives (NSO), Nuclear Safety Requirements (NSR), DAFMAN 91-119 requirements, verification procedures, and associated results.

Distribution Statement A. Approved for public release: distribution is unlimited.

Source: https://assist.dla.mil -- Downloaded: 2026-10-26T11:03Z
Check the source to verify that this is the current version before use.

c. This DID contains the content and intended use information for the data deliverable resulting from the work task described in the solicitation and defined in Chapter 4 of DAFMAN 91-119.

d. If the IVO did not provide a completed NSSVP to AFSEC/SEWN before the IVO performed the finalized verification activities, AFSEC/SEWN may not accept the results of the verification activities presented in the NSSVR.

e. All NSO and NSR examples listed in the document are examples of how an IVO could meet the requirements in DAFMAN 91-119. AFSEC/SEWN does not intend the examples to be directive. The definitions of NSO and NSR in Chapter 4 of DAFMAN 91-119 supersede this document.

f. The U.S. Navy is excluded from requirements set forth in this DID.

Source: https://assist.dla.mil -- Downloaded: 2026-10-26T11:03Z
Check the source to verify that this is the current version before use.

Requirements:

1 **Reference documents**. The contract shall specify the applicable issue of the documents cited herein, including their approval dates and dates of any applicable amendments, notices, and revisions.

2 **Format**. The NSSVR shall be in contractor's format (e.g., website, word processor file, or spreadsheet) for delivery. The NSSVR format shall contain tables as required in 3. If the contractor chooses to provide the NSSVR as a website, the website or a snapshot of the complete website must be viewable locally.

2.1 The latest NSSVP shall be delivered with the completed NSSVR. If the IVO has made any updates to the NSSVP due to the execution of the verification activities or the creation of the NSSVR, the NSSVR shall contain disclosure of those updates.

2.2 The NSSVR is not limited in length but is limited in scope per the Content portion of this DID.

2.3 The items described within the Content portion of this DID do not need to be in the same document. The contractor may separate the NSSVR into documents and attachments for ease of delivery. In particular, the verification procedures and requirements traceability could be delivered in spreadsheet attachments.

2.4 The NSSVR shall meet all requirements for NSSVRs found in DAFMAN 91-119. This includes restrictions found in DAFMAN 91-119 stating that the IVO cannot share the NSSVR with the DO without permission of AFSEC/SEWN.

3 **Content**. The NSSVR shall only contain the information relevant to NSDC of the software as DAFMAN 91-119 and DAFI 91-101 describe. In particular, the NSSVR shall not contain information for the development of Weapon System Safety Rules.

3.1 If the IVO separates the NSSVR into multiple documents to ease delivery described in 2.3 of this DID, the NSSVR shall include an overview of the internal organization of the document. If the NSSVR is a static snapshot of a website, the overview can be a sitemap.

3.2 The NSSVR shall describe in detail the nuclear weapon system that the IVO contractor is verifying. Except for clarity additions, as defined in the following bulleted content areas, this NSSVR nuclear weapon system description shall be a duplicate of the description the latest NSSVP provided.

3.2.1 Changes to the description of the nuclear weapon system.

3.2.2 Changes to the described breakdown of the nuclear weapon system components, especially the software components.

Source: https://assist.dla.mil -- Downloaded: 2026-10-26T11:03Z
Check the source to verify that this is the current version before use.

3.2.3 Changes to planned software component verification methodology, either Nuclear Safety Cross-Check Analysis (NSCCA) or Nuclear Safety Analysis and Technical Evaluation (NSATE).

3.3 The NSSVR shall define the general organization of the IVO including all supporting contractors. Except for clarity additions, as defined in the following bulleted content areas, this NSSVR IVO organizational definition shall be a duplicate of the definition the latest NSSVP provided.

3.3.1 Changes to the general organization of the team and IVO parent organization(s)

3.3.2 Changes to any contractors that are performing verification efforts specifically for the nuclear weapon system.

3.3.3 Changes to any organizations within the IVO that performs work specifically for NSDC software verification described in 3.2 of this DID.

3.4 For each organization included in the IVO (as 3.3 of this DID identifies), the NSSVR shall duplicate 3.5, 3.6, and 3.7 of this DID.

3.5 The NSSVR shall define the DO and explain interactions between the DO and the IVO. Except for clarity additions, these NSSVR DO/IVO interaction explanations shall be a duplicate of the interaction explanations provided in the latest NSSVP.

3.6 As defined in the latest NSSVP, the NSSVR shall identify Managerial Independence, Financial Independence, or Technical Independence issues that occurred during the IVO verification effort. The NSSVR shall provide evidence of the compensatory measures taken to resolve any issues and report on the result of those efforts.

3.7 In accordance with the Managerial Independence, Financial Independence, and Technical Independence defined in the latest NSSVP, the NSSVR shall provide answers to the following questions.

3.7.1 Did the system program office try to project undue influence on cost and schedule of the verification activities? Undue influence can come in many forms from schedule based financial incentives, budget limitations, to specific completion deadlines.

3.7.2 What process did the IVO use to acquire software development and verification tools and how did the process ensure independence from the DO for each tool acquisition?

3.7.3 What software development and verification tools did the IVO acquire from the DO for the verification environment, and why?

Source: https://assist.dla.mil -- Downloaded: 2026-10-26T11:03Z
Check the source to verify that this is the current version before use.

3.7.4 What processes did the IVO use to develop the verification suite independently from the DO?

3.7.5 If the IVO is performing NSCCA, did the IVO identify the verification requirements and procedures that actively seek out sabotage or unintended actions that could violate nuclear safety? What measures did the IVO take to ensure that the IVO developed these verification requirements and procedures independently from the DO?

3.7.6 What preventative measures did the IVO take to avoid providing any methods, verification plans, configurations, or verification-related data to the DO?

3.7.7 What measures did the IVO take to ensure independence when the DO and the IVO shared limited resources such as hardware integration laboratories?

3.7.8 Did the independently assembled development and verification suite match the DO's tool suite? If not, why.

3.8 If NSCCA configuration management system and security and control measures were defined in the latest NSSVP and changes occurred during the verification effort, the NSSVR shall provide evidence of the compensatory measures taken to resolve the issues and report on the result of those efforts.

3.9 The NSSVR shall provide answers to the following questions pertaining to the IVO's NSATE and NSCCA verification efforts.

3.9.1 Did the IVO use data representing real-world operational scenarios in their verification efforts? This applies only to verification procedures that require scenarios during execution, such as procedures that simulate the release of a weapon.

3.9.2 Did the IVO create verification procedures to ensure that the source code includes error handling that detects and appropriately responds to Critical Signal-impacting hardware and software faults during all stages of execution?

3.9.3 Did the IVO execute verification procedures on production-equivalent platform hardware? This applies only to verification procedures that require execution of operational software, such as procedures verifying the actions of the stores management system.

3.9.4 Did the DO provide all necessary data that the IVO needed to verify that the software configuration is fully compliant with nuclear surety requirements?

3.9.5 How did the IVO evaluate and verify the software source code and associated software executable version(s)? This applies to the compilation and linking

Source: https://assist.dla.mil -- Downloaded: 2026-10-26T11:03Z
Check the source to verify that this is the current version before use.

process and the IVO responsibility to ensure that the source code compiles and links to the executable. For scope clarification, auto-generated code is considered software source code.

3.9.6 Did the IVO request any additional software components beyond those originally considered for verification, particularly software components that interface with those software components originally considered for verification, due to concerns that the interfacing software had a nuclear safety impact? What additional software components did the IVO identify and how did the IVO verify those components in the applicable verification procedures?

3.9.7 When the IVO used the DO tools to verify the DO's test results, did the IVO achieve identical results? Can the IVO explain any differing results?

3.10 The NSSVR shall provide answers to the following questions pertaining to the IVO's NSCCA verification efforts.

3.10.1 For verification of modifications made to previously NSCCA-certified software, did the IVO verify that all interfacing system components did not have any potential critical signal impacts? What interfacing system components did the IVO identify?

3.10.2 Did the IVO identify NSCCA-specific verification activities? Did the verification results show that the system only performed authorized functions when it receives authorization to perform those functions, specifically functions related to Critical Signals?

3.11 The NSSVR shall describe the development and verification software tools the IVO used during the verification effort, both created and acquired by the IVO. Refer to listed information in 3.11.1 through 3.11.4 (see Table 1). Except for clarity additions, as defined in the following bulleted content areas, the NSSVR software tool listing shall be a duplicate of the listing provided in the latest NSSVP.

3.11.1 Name of software tool.

3.11.2 Software tool creator or manufacturer.

3.11.3 General description of how the IVO used the software tool.

3.11.4 If the DO provided the software tool, explanations of how the IVO ensured technical independence of this tool.

3.12 The NSSVR shall describe the NSOs the IVO developed and how the NSOs trace to DAFMAN 91-119 (see Table 2). The definition of NSO is in Chapter 4 of DAFMAN

Source: https://assist.dla.mil -- Downloaded: 2026-10-26T11:03Z
Check the source to verify that this is the current version before use.

91-119. The NSSVR NSO to DAFMAN 91-119 trace shall be a duplicate of the latest NSSVP trace.

3.13 The NSSVR shall describe the NSRs the IVO developed and how the NSRs trace to the NSOs. The definition of NSR is in Chapter 4 of DAFMAN 91-119 (see Table 3). The NSSVR NSR to NSO trace shall be a duplicate of the NSSVP trace.

3.14 The NSSVR shall include a summary of all the verification procedures that includes an overview of the procedure and tracing to the appropriate NSR (see Table 4). The NSSVR NSRs → verification procedure summary trace shall be a duplicate of the NSSVP trace.

3.15 The NSSVR shall include the full set of verification procedures within the greater document or as a separate attachment. Each procedure shall reference the NSR to which the procedure traces as well as verification type(s) – Test, Demonstration, Inspection, Analysis. The NSSVR NSR → verification procedure → verification type trace shall be a duplicate of the NSSVP trace.

3.16 The NSSVR shall include a full traceability analysis flowing from the DAFMAN 91-119 requirements → NSOs → NSRs → Verification procedures and the reverse trace flowing from the Verification procedures → NSRs → NSOs → DAFMAN 91-119 requirements. The traceability analysis shall be easily navigable to ease the search for untraced objects within the analysis. The analysis may be a static snapshot of a website with links showing connections from one level to the next (see Table 5 and Table 6). The NSSVR full traceability analysis shall be a duplicate of the NSSVP full traceability analysis.

3.17 The NSSVR shall include a listing of the verification results for all verification procedures. The listing shall trace from the NSRs → verification procedure summaries → verifier name or identifier → pass/fail results (see Table 7).

3.18 The NSSVR shall include a listing of all failed verification procedures. The listing shall contain verification procedure summary, verifier name or identifier, failure explanation, resolution, and comments (see Table 8).

Source: https://assist.dla.mil -- Downloaded: 2026-10-26T11:03Z
Check the source to verify that this is the current version before use.

Tool Name	Creator/ Manufacturer	General Description of Use	Created by DO?	Reason for Confidence in DO tools
XYZ Static Analysis Tool	XYZ, Inc.	Analysis tool to identify coding standard violations, including analysis of MISRA C and C++ standards.	No	N/A
Company A Build Script	Company A	Script used to build DO software into object code for both test environment and target hardware.	Yes	SW Team reviewed script and associated libraries, analyzed output of build for both test and target, executed cybersecurity tools on output to look for indicators of unexpected code insertion.

TABLE 1. Sample software tool entry.

DAFMAN 91-119 Requirement	NSO
8.15.1. The software shall not contain unused code.	1.1.1 The verification process shall prove that the software does not contain code that is untestable except for justifiable circumstances.
8.15.2. The software shall not contain global variables.	1.1.2 The verification process shall prove that the software does not contain global variables.
8.15.5. The software shall not execute memory allocation functions after application initialization.	1.1.3 The verification process shall prove that there is no portion of the software that executes “new” or “malloc” functionality once the initAllTasks function is complete.

TABLE 2. Sample NSO entry.

NSO	NSR
1.1.1 The verification process shall prove that the software does not contain code that is untestable except for justifiable circumstances.	1.1.1.A – The verifier shall execute XYZ Static Analysis Tool to identify obvious non-executable sections of code. If the tool identifies any sections of code as non-executable, the verifier shall analyze the sections to determine if the identification is a false positive. The verifier shall request another verifier perform the same analysis on identified sections. 1.1.1.B – The verifier shall execute dynamic testing using ABC code coverage tool to identify non-executable sections of code. If the tool identifies any sections of code as non-executable, the verifier shall analyze the sections to determine if the identification is a false positive. The verifier shall request another verifier perform the same analysis on identified sections.

Source: https://assist.dla.mil -- Downloaded: 2026-10-26T11:03Z
Check the source to verify that this is the current version before use.

NSO	NSR
	1.1.1.C – If the verifiers identify any sections of code as non-executable using the above steps and the verifiers have shown that the identifications are not a false positive, the verifiers shall analyze the identified sections to determine if the software section falls under a justifiable exemption (nature of the code, error handling in the code or low-level source code the software developer designed for very specific scenarios.) 1.1.1.D – If the verifier cannot instrument a section of code for dynamic analysis, the verifier shall analyze the section of code to determine if there are un-executable sections. If the analysis identifies any sections of code as non-executable, the verifier shall request another verifier perform the same analysis on identified sections. The verifiers shall then meet NSR 1.1.1.C.
1.1.2 The verification process shall prove that the software does not contain global variables.	1.1.2.A – The verifier shall execute XYZ Static Analysis Tool to identify global variable or global constant declarations, including any extern declarations, using the following definition of global: An object that allows direct data manipulation across different segments or functional sections without using data encapsulation methods. If the tool identifies any variable, functions, or declarations as global, the verifier shall analyze the sections to determine if the identification is a false positive. The verifier shall request another verifier perform the same analysis on identified sections.
1.1.3 The verification process shall prove that there is no portion of the software that executes “new” or “malloc” functionality once the initAllTasks function is complete.	1.1.3.A The verifier shall use the built-in operating system tool to identify any memory allocation that occurs after the initAllTasks function is complete. If the tool identifies any unacceptable memory allocations, the verifier shall analyze the software to determine if the identification is a false positive. The verifier shall request another verifier perform the same analysis on identified software.

TABLE 3. Sample NSR entry.

NSR	Verification Procedure Summary
1.1.1.A - The verifier shall execute XYZ Static Analysis Tool to identify obvious non-executable sections of code. If the tool identifies any sections of code as non-executable, the verifier shall analyze the sections to determine if the identification is a false positive. The verifier shall request another verifier perform the same analysis on identified sections.	Procedure 13 – Steps to identify obvious non-executable code and reject false positives through analysis.
1.1.2.A – The verifier shall execute XYZ Static Analysis Tool to identify global variable or global constant	Procedure 42 – Steps to identify global variables, constants, and

Source: https://assist.dla.mil -- Downloaded: 2026-10-26T11:03Z
Check the source to verify that this is the current version before use.

NSR	Verification Procedure Summary
declarations, including any extern declarations, using the following definition of global: An object that allows direct data manipulation across different segments or functional sections without using data encapsulation methods. If the tool identifies any variable, functions, or declarations as global, the verifier shall analyze the sections to determine if the identification is a false positive. The verifier shall request another verifier perform the same analysis on identified sections.	externs, and reject false positives through analysis.
1.1.3.A The verifier shall use the built-in operating system tool to identify any memory allocation that occurs after the initAllTasks function is complete. If the tool identifies any unacceptable memory allocations, the verifier shall analyze the software to determine if the identification is a false positive. The verifier shall request another verifier perform the same analysis on identified software.	Procedure 143 – Steps to identify memory allocations after initialization is complete and reject false positives through analysis.

TABLE 4. Sample verification procedure summary entry.

DAFMAN 91-119 Requirement	NSO	NSR	Verification Procedure
8.15.1. The software shall not contain unused code.	1.1.1 The verification process shall prove that the software does not contain code that is untestable except for justifiable circumstances.	1.1.1.A – The verifier shall execute XYZ Static Analysis Tool to identify obvious non-executable sections of code. If the tool identifies any sections of code as non-executable, the verifier shall analyze the sections to determine if the identification is a false positive. The verifier shall request another verifier perform the same analysis on identified sections.	Procedure 13
8.15.2. The software shall not contain global variables.	1.1.2 The verification process shall prove that the software does not contain global variables.	1.1.2.A – The verifier shall execute XYZ Static Analysis Tool to identify global variable or global constant declarations, including any extern declarations, using the following definition of global: An object that allows direct data manipulation across different segments or	Procedure 42

Source: https://assist.dla.mil -- Downloaded: 2026-10-26T11:03Z
Check the source to verify that this is the current version before use.

DAFMAN 91-119 Requirement	NSO	NSR	Verification Procedure
		functional sections without using data encapsulation methods. If the tool identifies any variable, functions, or declarations as global, the verifier shall analyze the sections to determine if the identification is a false positive. The verifier shall request another verifier perform the same analysis on identified sections.	
8.15.5. The software shall not execute memory allocation functions after application initialization.	1.1.3 The verification process shall prove that there is no portion of the software that executes “new” or “malloc” functionality once the initAllTasks function is complete.	1.1.3.A The verifier shall use the built-in operating system tool to identify any memory allocation that occurs after the initAllTasks function is complete. If the tool identifies any unacceptable memory allocations, the verifier shall analyze the software to determine if the identification is a false positive. The verifier shall request another verifier perform the same analysis on identified software.	Procedure 143

TABLE 5. Sample verification forward trace entry.

Verification Procedure	NSR	NSO	DAFMAN 91-119 Requirement
Procedure 13	1.1.1.A – The verifier shall execute XYZ Static Analysis Tool to identify obvious non-executable sections of code. If the tool identifies any sections of code as non-executable, the verifier shall analyze the sections to determine if the identification is a false positive. The verifier shall request another verifier perform the same analysis on identified sections.	1.1.1 The verification process shall prove that the software does not contain code that is untestable except for justifiable circumstances.	8.15.1. The software shall not contain unused code.

Source: https://assist.dla.mil -- Downloaded: 2026-10-26T11:03Z
Check the source to verify that this is the current version before use.

Verification Procedure	NSR	NSO	DAFMAN 91-119 Requirement
Procedure 42	1.1.2.A – The verifier shall execute XYZ Static Analysis Tool to identify global variable or global constant declarations, including any extern declarations, using the following definition of global: An object that allows direct data manipulation across different segments or functional sections without using data encapsulation methods. If the tool identifies any variable, functions, or declarations as global, the verifier shall analyze the sections to determine if the identification is a false positive. The verifier shall request another verifier perform the same analysis on identified sections.	1.1.2 The verification process shall prove that the software does not contain global variables.	8.15.2. The software shall not contain global variables.
Procedure 143	1.1.3.A The verifier shall use the built-in operating system tool to identify any memory allocation that occurs after the initAllTasks function is complete. If the tool identifies any unacceptable memory allocations, the verifier shall analyze the software to determine if the identification is a false positive. The verifier shall request another verifier perform the same analysis on identified software.	1.1.3 The verification process shall prove that there is no portion of the software that executes “new” or “malloc” functionality once the initAllTasks function is complete.	8.15.5. The software shall not execute memory allocation functions after application initialization.

TABLE 6. Sample verification reverse trace entry.

NSR	Verification Procedure Summary	Verifier	Verification Result
1.1.1.A - The verifier shall execute XYZ Static Analysis Tool to identify obvious non-executable sections of code. If the tool identifies any sections of code as non-executable, the verifier shall analyze the sections to determine if the identification is a	Procedure 13 – Steps to identify obvious non-executable code and reject false	Inspector ABC, Inspector DEF	Fail

Source: https://assist.dla.mil -- Downloaded: 2026-10-26T11:03Z
Check the source to verify that this is the current version before use.

NSR	Verification Procedure Summary	Verifier	Verification Result
false positive. The verifier shall request another verifier perform the same analysis on identified sections.	positives through analysis.		
1.1.2.A – The verifier shall execute XYZ Static Analysis Tool to identify global variable or global constant declarations, including any extern declarations, using the following definition of global: An object that allows direct data manipulation across different segments or functional sections without using data encapsulation methods. If the tool identifies any variable, functions, or declarations as global, the verifier shall analyze the sections to determine if the identification is a false positive. The verifier shall request another verifier perform the same analysis on identified sections.	Procedure 42 – Steps to identify global variables, constants, and externs, and reject false positives through analysis.	Inspector ABC	Pass
1.1.3.A The verifier shall use the built-in operating system tool to identify any memory allocation that occurs after the initAllTasks function is complete. If the tool identifies any unacceptable memory allocations, the verifier shall analyze the software to determine if the identification is a false positive. The verifier shall request another verifier perform the same analysis on identified software.	Procedure 143 – Steps to identify memory allocations after initialization is complete and reject false positives through analysis.	Inspector ABC, Inspector DEF	Fail

TABLE 7. Sample verification results trace entry.

Verification Procedure Summary	Verifier	Failure Explanation	Resolution	Comments
Procedure 13 – Steps to identify obvious non-executable code and reject false positives through analysis.	Inspector ABC, Inspector DEF	Multiple legacy processing functions were found using XYZ Static Analysis Tool and Inspector ABC and Inspector DEF confirmed the finding by visual code inspection.	IVO recommends that AFSEC/SEWN Inform the DO of the discovery for removal of the identified	

Source: https://assist.dla.mil -- Downloaded: 2026-10-26T11:03Z
Check the source to verify that this is the current version before use.

Verification Procedure Summary	Verifier	Failure Explanation	Resolution	Comments
			unused function code.	
Procedure 42 – Steps to identify global variables, constants, and externs, and reject false positives through analysis.	Inspector ABC, Inspector DEF	Multiple legacy global variables and constants were found using XYZ Static Analysis Tool.	Inspector ABC and Inspector DEF performed visual code inspection and found that the variables and constants were locally isolated. Global parameter adjustments were made to the XYZ Static Analysis Tool resulting in a Pass result matching visual inspection.	IVO verification team have recorded this false-positive event in the verification tool findings log.
Procedure 143 – Steps to identify memory allocations after initialization is complete and reject false positives through analysis.	Inspector ABC, Inspector DEF	Using the built-in operating system tool, Inspector ABC identified multiple memory allocations occurred 30 seconds after initAllTasks function completed. Inspector DEF reran Test Procedure 143 and verified the same failure observation.	IVO recommends that AFSEC/SEWN inform the DO of the discovery of the extra memory allocation finding.	

TABLE 8. Sample failed verification procedures entry.

End of DI-NUOR-82430A

Source: https://assist.dla.mil -- Downloaded: 2026-10-26T11:03Z
Check the source to verify that this is the current version before use.